

To build **Shabari** robustly using **Manus AI**, we will follow a high-precision, multi-step engineering plan.

As requested, here is the **Step 1 Prompt**, refined with 2026-specific Indian regulations (including DPDP Act compliance, NPCI transaction limits, and the latest VPA verification standards).

Step 1: The "Intelligence" & Risk Engine

Copy and paste this into Manus AI:

"Act as a Lead Fintech Engineer. Build the core backend and fraud-detection logic for 'Shabari', a mobile security app for the Indian market, ensuring compliance with the **DPDP Act 2023** and **2026 NPCI/RBI guidelines**.

1. Core Logic & UPI Interceptor

- **Intent Parser:** Implement a Python module to parse `upi://pay` strings. Specifically detect 'Collect Request' fraud by flagging any URI that contains an `am` (amount) parameter but is contextually presented as a 'Refund' or 'Cashback' (check `tn` and `pn` parameters).
- **NPCI Compliance:** Enforce a logic check for P2P collect requests; if the amount exceeds ₹2,000 (per 2026 NPCI limits), flag it as a 'Protocol Violation'.

2. VPA & Identity Verification (Penny-Less)

- **Identity Engine:** Build a module to interface with a VPA Verification API (e.g., Cashfree/Razorpay). Instead of a slow Penny Drop, use the 'Validate Address API' to fetch the **Official Bank Registered Name** associated with a VPA instantly.
- **Mismatch Logic:** If the name in the QR/Link (`pn`) significantly differs from the Bank Name, assign a high risk score.

3. Link & Malware Scanner

- **Phishing Detection:** Integrate Google Safe Browsing API.
- **Malware Analysis:** Integrate VirusTotal API specifically for `.apk` and `.zip` download links.
- **The Unshortener:** Implement a service that resolves shortened URLs (bit.ly, tinyurl) to their final destination before scanning.

4. 2026 Weighted Risk Scorer (0-100)

- **Algorithm:** Create a scoring function based on:
 - **Reputation (40%):** Matches against a blacklist of known fraudulent VPAs/URLs.
 - **Heuristics (30%):** Detection of 'Collect-as-Receive' patterns or suspicious keywords.
 - **Identity (30%):** Name mismatch or newness of the VPA.

5. Privacy-First Database (DPDP Compliant)

- **Schema:** Use PostgreSQL for the master fraud DB.
- **Data Minimization:** Do NOT store plain-text VPAs. Store only **SHA-256 hashes** of malicious entities to protect user privacy.
- **Local Cache:** Implement a **Bloom Filter** or Redis cache for 'Hot-List' scammers to allow the mobile app to check threats in <50ms.

6. Tech Stack

- **Framework:** FastAPI (Python).
- **Database:** PostgreSQL + Redis.
- **Security:** Implement TLS 1.3 for all API communication."

What this Step 1 achieves:

- **Speed:** The Bloom Filter ensures your app can warn users almost instantly.
- **Cost Efficiency:** Using VPA Lookups instead of Penny Drops saves you money during the MVP phase.
- **Legality:** By hashing the data, you comply with India's strict **DPDP Act** privacy laws from day one.

This is the most critical step for user trust. We are building the "**Active Shield**"—the interface that the user sees when they are about to make a mistake. Since it is 2026, we must leverage **Android 14/15/16** features like "Notification Listener" and "App Links" to catch scams without being invasive.

Step 2: The "Security UX & Shield Interface"

Copy and paste this into Manus AI:

"Build the mobile frontend and real-time intervention layer for 'Shabari' using **Flutter** (or Native Kotlin). This module must handle the 'Shabari-First' scanning logic and the 'Friction' UI.

1. App Manifest & High-Trust Permissions

- **Permissions:** Request `CAMERA`, `POST_NOTIFICATIONS`, and `BIND_NOTIFICATION_LISTENER_SERVICE`.
- **Intent Filters:** Register the app to handle `upi://pay` and `https://` schemes. Ensure **App Links** are configured so Shabari is the default 'Safe Gate' for financial URLs clicked in WhatsApp or SMS.

2. 'Shabari-First' QR Scanner

- **Scanner:** Build a high-speed QR scanner (using `mobile_scanner` or `camera` package).
- **The Workflow:** > * When a QR is scanned, do NOT open the bank app.
 - Send the string to the Step 1 Backend API for a **Trust Score**.
 - Show a 'Verifying...' shimmer effect while the backend fetches the **Bank Registered Name**.

3. 3-Tier Friction UI (The Shield)

- **Green (Trust Score 80-100):** Show a 'Verified Merchant' badge with the bank-registered name. Provide a large 'Pay via GPay/PhonePe' button that triggers a UPI Intent.
- **Yellow (Trust Score 40-79):** Show a 'Caution' banner. Highlight if the 'Merchant Name' on the QR does not match the 'Bank Name'.
- **Red (Trust Score 0-39):** > * Launch a full-screen **Security Intervention**.
 - **The 5-Second Lockdown:** Disable the 'Proceed' button for 5 seconds.
 - Display a bold warning: *'STOP: This account is linked to Job Fraud / Electricity Bill Scams. If you enter your PIN, you will LOSE money.'*

4. Notification Listener (The 'Silent Guard')

- **Service:** Implement a `NotificationListenerService` that monitors incoming notifications from payment apps (PhonePe, GPay, Paytm).
- **Logic:** If a notification contains 'Collect Request' or 'Requested Money' from an unknown or blacklisted VPA, fire a **Shabari High-Priority Alert** notification immediately to warn the user: '⚠️ *Do not tap! This is a fake refund request.*'

5. Dynamic Guidance UX

- **Overlay:** For high-risk links clicked outside the app, use a **System Alert Window** (Overlay) to display a 'Risk Summary' before the user enters the target website.
- **The 'Shabari-Verified' Badge:** Implement a floating 'Safe Scan' widget that users can toggle to quickly scan any screen-captured QR code from their gallery."

Why this Step 2 is Robust:

- **The Interceptor:** By registering as a `upi://` handler, your app becomes the "Firewall" for every financial link the user clicks.
- **Notification Guard:** This is the *only* way to stop "Collect Request" scams where the user is tricked into tapping a notification and entering their PIN.
- **Psychological Friction:** The 5-second timer is a proven way to break the "Panic Loop" that scammers create.

Step 3 of building **Shabari** focuses on the **"Identity & Hardware Shield."** In 2026, the Department of Telecommunications (DoT) has made **SIM binding** mandatory for all communication and financial apps. This module ensures that even if a scammer steals a user's OTP, they cannot access the account because the physical SIM is missing.

Step 3: The "Identity & Hardware Shield"

Copy and paste this into Manus AI:

"Act as a Mobile Security Architect. Implement the hardware-level security and government integration module for 'Shabari' to comply with the **2026 Telecom Cyber Security Rules**.

1. Mandatory SIM Binding (Hardware Lock)

- **Baseline Capture:** On the first successful login, use the `SubscriptionManager` API to capture and securely store the **ICCID** (SIM Serial Number) and **IMSI** (Subscriber ID).
- **Persistent Heartbeat:** Create a `WorkManager` task or a `Service` that triggers on every app launch and whenever the `SIM_STATE_CHANGED` broadcast intent is received.
- **Integrity Check:** Compare the current ICCID with the stored 'Trusted SIM' ID. If a mismatch is detected (SIM Swap) or if the SIM slot is empty, immediately trigger the **Cooling Period State**.

2. The 24-Hour 'Cooling Period' Logic

- **State Management:** Implement a local and server-side flag `is_in_cooling_period`.
- **Restriction UI:** When active, the app must:
 - Disable all 'Scan & Pay' and 'Verify Link' features.
 - Show a persistent **Security Dashboard** with a 24-hour countdown timer.
 - Block the launch of any UPI Intent from within Shabari.
- **Emergency Recovery:** Provide a 'Re-verify Identity' flow that requires **Liveness Detection** (a video selfie) or a face scan compared against the user's initial setup.

3. Sanchar Saathi (Identity Hygiene) Integration

- **Audit Tool:** Build a dedicated 'Identity Health' screen.
- **TAF-COP Module:** Add a button that launches the **Sanchar Saathi / TAF-COP** portal in a **Chrome Custom Tab** (using <https://tafcop.sancharsaathi.gov.in/>).
- **Self-Reporting Guide:** If the user identifies unknown SIMs, provide an in-app wizard that helps them draft the 'Not My Number' report for the DoT portal.

4. Telecommunication Identifier User Entity (TIUE) Compliance

- **Logout Logic:** Implement the 2026 mandatory **6-hour periodic logout** for any web instance of the app.

- **Traceability:** Ensure every security report generated by the app includes a cryptographically signed timestamp and the 'SIM-Device' binding ID to aid law enforcement if a fraud is reported.

5. Technical Requirements

- **Permissions:** Request `READ_PHONE_STATE` and `ACCESS_COARSE_LOCATION` (required by some Android versions for SIM info).
- **Storage:** Use **Android Keystore** to store the ICCID hashes so they cannot be accessed by other malicious apps or via a rooted device."

Why this Step 3 completes the robustness:

- **Anti-Swap:** By checking the **ICCID** (hardware ID) rather than just the phone number, you stop the #1 technique used in high-value banking thefts.
- **Traceability:** Anchoring the app to a live, KYC-verified SIM ensures that Shabari remains a "trusted entity" under Indian law.
- **Identity Empowerment:** Integrating Sanchar Saathi doesn't just protect the app; it protects the user's entire Aadhaar-linked identity.